The EFGP-SING E-step without Monte Carlo

Closed form via a custom Gaussian-mixture spreader

Abstract

The EFGP-SING E-step needs, for each output dim r, the marginal $q(x)$-expectations that build the precision A_r and right-hand side h_r (Eqs. (2)–(4)). This note makes one observation: every such expectation is an instance of $\mathbb{E}_q[\sum_i w_i\, e^{2\pi i\, x_i^\top \xi}]$, the expectation of a nonuniform Fourier sum over the latent points, and *because expectation and the Fourier transform are both linear and commute*, this equals the Fourier transform of the *expected* source density $g(x) = \sum_i w_i\, \mathcal{N}(x; m_i, S_i)$. The entire E-step is therefore "deposit g on a grid and FFT it." No per-source characteristic function is ever evaluated — the FFT performs the averaging; the Gaussian CF is merely the per-source *description* of the same target, useful for reading off structure (BTTB, the cheap derivative rule) but never a computational ingredient.

This is realised as a custom Gaussian-mixture spreader, now the default EFGP-SING E-step (`estep_method='gmix'`), replacing the earlier Monte-Carlo build (a pseudo-cloud $x_i^{(s)} \sim q(x_i)$ through a single Type-1 NUFFT; see `efgp_estep.tex`). It is deterministic (no MC noise into the M-step), preserves the BTTB structure of A_r, and approximates the target by equispaced quadrature — its only errors are FFT discretisation and Gaussian-stencil truncation, both controllable. Cost is $O(T \cdot (\sigma_{\max}/\sigma_{\min})^D + N^D \log N)$ per FFT-set call. Empirically, on a GP-drift recovery benchmark with fixed emissions, it recovers ℓ and σ^2 slightly closer to truth than the $S = 2$ MC default, yields slightly lower drift error, and — unlike MC — has no catastrophic-divergence failure mode at large T (§9).

Contents

1 The expectations we need

We adopt the notation of `efgp_estep.tex` (Layers 1–3). The mean-field optimum for $q(w_r) = \mathcal{N}(\mu_r, A_r^{-1})$ is the system

$$A_r\,\mu_r \;=\; h_r, \tag{1}$$

with

$$A_r \;=\; I + \frac{1}{\sigma_r^2}\sum_i \Delta_i\,\mathbb{E}_{q(x_i)}\big[\phi(x_i)\phi(x_i)^*\big], \qquad h_r \;=\; \frac{1}{\sigma_r^2}\sum_i \mathbb{E}_{q(x_i,\,x_{i+1})}\big[\phi(x_i)\,(x_{i+1,r} - x_{i,r})\big], \tag{2}$$

and Fourier features

$$\phi_\theta(x) \;=\; D_\theta\,F_x, \qquad F_x[k] \;=\; e^{2\pi i\,x^\top \xi_k}. \tag{3}$$

Stein's lemma collapses the pairwise expectation in h_r to marginal expectations under $q(x_i)$ plus a covariance correction:

$$h_r \;=\; \frac{1}{\sigma_r^2}\sum_i \Big\{\mathbb{E}_{q(x_i)}[\phi(x_i)]\,d_{i,r} \,+\, \mathbb{E}_{q(x_i)}[J_\phi(x_i)]^*\,C_{i,\cdot r}\Big\}, \qquad d_i := m_{i+1} - m_i,\;\; C_i := S_{i,i+1} - S_i. \tag{4}$$

Every quantity in (2)–(4) is a *marginal* expectation under $q(x_i) = \mathcal{N}(m_i, S_i)$ of a complex exponential or its derivative. These are all instances of one primitive — the expectation of a Fourier sum — which §2 collapses to a single FFT of a density.

2 The one primitive: $\mathbb{E}$[Fourier sum] is an FFT of a density

Collect the quantities of §1 into a single object: each is

$$F(\xi) \;=\; \mathbb{E}_q\Big[\sum_i w_i\,e^{2\pi i\,x_i^\top \xi}\Big], \tag{5}$$

for some weights w_i and output frequencies ξ on a regular grid (the spectral grid $\{\xi_k\}$ for h_r, the difference grid $\{\xi_k - \xi_\ell\}$ for A_r). The bracketed sum is a nonuniform Fourier sum over the random points $\{x_i\}$ — a Type-1 NUFFT of point masses — and the whole E-step reduces to taking its *expectation*.

Expectation commutes with the Fourier transform. Expectation is an integral and the Fourier transform is an integral, so the two commute: $\mathbb{E}$ slides *through* the transform and lands on the source distribution rather than on each exponential. Step by step — take the expectation inside the sum, write each $\mathbb{E}[e^{2\pi i\, x_i^\top \xi}] = \int \mathcal{N}(x; m_i, S_i)\, e^{2\pi i\, \xi^\top x}\, dx$ as the Fourier transform of the i-th density (an integral, no closed form needed), and pull the sum back inside:

$$\boxed{F(\xi) \;=\; \sum_i w_i\, \mathbb{E}\big[e^{2\pi i\, x_i^\top \xi}\big] \;=\; \int \Big(\underbrace{\textstyle\sum_i w_i\, \mathcal{N}(x; m_i, S_i)}_{=:\, g(x)} \Big) e^{2\pi i\, \xi^\top x}\, dx \;=\; \widehat{g}(\xi).} \tag{6}$$

The density g is the *expected point cloud* — each random point replaced by its Gaussian smear:

$$g(x) \;=\; \mathbb{E}_q\Big[\sum_i w_i\, \delta(x - x_i)\Big] \;=\; \sum_i w_i\, \mathcal{N}(x;\, m_i, S_i). \tag{7}$$

So the entire E-step primitive is one instruction: *deposit the expected density g on a grid and FFT it*, $\widehat{g}(\xi) \approx \mathrm{FFT}[g]$ on the dual grid (§6). Nothing per-source is expanded in closed form; the FFT performs the averaging, because it is linear.

Same target as Monte Carlo, two commuting operations in the other order. This is also the exact relationship to the MC E-step (`efgp_estep.tex`) that `estep_method='gmix'` replaces:

- **MC**: *sample* the cloud, then FFT — $\frac{1}{S}\sum_s \mathrm{NUFFT}\big[\sum_i w_i\, \delta(\cdot - x_i^{(s)})\big]$.
- **gmix**: take the *expectation first* ($\to g$), then FFT.

They target the identical F and agree precisely because $\mathbb{E}$ and FFT commute; gmix does the averaging analytically by depositing g, so it is deterministic — no shot noise enters the M-step.

Where the characteristic function hides. The per-source factor $\mathbb{E}_{e^{2\pi i\, x_i^\top \xi}}$ that (6) left as an integral has the closed-form value $e^{2\pi i\, m^\top \xi}\, e^{-2\pi^2\, \xi^\top S\, \xi}$ — the Gaussian characteristic function. Substituting it (and reattaching the feature normaliser D_θ, using $\partial_{x_j} \phi_k = 2\pi i\, \xi_{k,j}\, \phi_k$) gives the equivalent *spectral-side* forms

$$\mathbb{E}_{q(x_i)}[\phi_k(x)] \;=\; D_{\theta,k}\, e^{2\pi i\, m^\top \xi_k}\, e^{-2\pi^2\, \xi_k^\top S\, \xi_k}, \tag{8}$$

$$\mathbb{E}_{q(x_i)}[\phi_k(x)\, \overline{\phi_\ell(x)}] \;=\; D_{\theta,k}\, \overline{D_{\theta,\ell}}\, e^{2\pi i\, m^\top (\xi_k - \xi_\ell)}\, e^{-2\pi^2\, (\xi_k - \xi_\ell)^\top S\, (\xi_k - \xi_\ell)}, \tag{9}$$

$$\mathbb{E}_{q(x_i)}\big[\partial_{x_j} \phi_k(x)\big] \;=\; 2\pi i\, \xi_{k,j}\, \mathbb{E}_{q(x_i)}[\phi_k(x)]. \tag{10}$$

These are a useful *description* of the target F — they expose the per-source damping $e^{-2\pi^2 \xi^\top S_i \xi}$, the BTTB structure (§3), and the cheap derivative rule (10) — but they are *never evaluated*. The algorithm computes F as $\widehat{g}$ by one FFT; the damping factor *emerges* as the transform of the deposited density, not from its closed form. The only closed form the code consumes is the Gaussian *density* in (7) (the spread weights), the other half of the same Fourier pair.

Two structural facts the spectral-side forms make visible. First, (9) is *not* the product of (8) with its conjugate — the damping sees $\xi_k - \xi_\ell$, not ξ_k and ξ_ℓ separately. This is the single most important consequence of x being uncertain, and it is what makes A_r BTTB in the frequency *differences* (§3). Second, the derivative rule (10) means the Stein-correction term in h_r is the *same* F multiplied by $2\pi i\, \xi_j$ on the spectral grid: one extra coefficient grid through the same FFT, with no new density and no per-source work.

3 The precision A_r: BTTB is preserved

From (2), the precision is built from $\mathbb{E}_{q(x_i)}[\phi_k(x)\overline{\phi_\ell(x)}]$, which is the expectation of a *single* exponential at the difference frequency:

$$\mathbb{E}_{q(x_i)}[\phi_k\overline{\phi_\ell}] \;=\; D_{\theta,k}\,\overline{D_{\theta,\ell}}\,\mathbb{E}_{q(x_i)}\big[e^{2\pi i\,x^\top(\xi_k-\xi_\ell)}\big]. \tag{11}$$

So it depends on (k,ℓ) only through $\Delta\xi := \xi_k - \xi_\ell$ — for *any* q, before any Gaussian assumption. Hence, on an equispaced spectral grid, $A_r = I + D_\theta T_r D_\theta$ is BTTB, exactly as in the MC formulation, with generator

$$\boxed{T_r[\Delta\xi] \;=\; \sum_i \frac{\Delta_i}{\sigma_r^2}\,\mathbb{E}_{q(x_i)}\big[e^{2\pi i\,x_i^\top\Delta\xi}\big] \;=\; \widehat{g_T}(\Delta\xi), \qquad g_T := \sum_i \frac{\Delta_i}{\sigma_r^2}\,\mathcal{N}(\cdot;m_i,S_i).} \tag{12}$$

This is exactly the §2 primitive (6) with weights $w_i = \Delta_i/\sigma_r^2$, evaluated on the difference grid: the FFT of one deposited density g_T. (The per-source expectation expands to the spectral-side CF $\sum_i(\Delta_i/\sigma_r^2)\,e^{2\pi i\,m_i^\top\Delta\xi}\,e^{-2\pi^2\,\Delta\xi^\top S_i\,\Delta\xi}$, but — as in §2 — that form is never evaluated.) Matvecs $A_r v$ remain $O(M\log M)$ via FFT on the generator; nothing about `make_A_apply` or the CG solver changes.

Why it is not a NUFFT of the means. The MC version forms T_r from a single Type-1 NUFFT of a sampled point cloud, $O(N + M\log M)$. The generator (12) is *not* a NUFFT of the means $\{m_i\}$: the sources are not points but the smeared density g_T, and the per-source spread of $\mathcal{N}(\cdot;m_i,S_i)$ — equivalently the damping $e^{-2\pi^2\,\Delta\xi^\top S_i\,\Delta\xi}$ — couples the output frequency $\Delta\xi$ and the source i together. Pulling it outside a single NUFFT of the means would require S_i independent of i. The resolution is precisely §2: it is the FFT of the expected density, carried out by the spreader of §6.

4 The right-hand side h_r

The same primitive builds h_r, now on the spectral grid $\{\xi_k\}$. The mean-increment term of (4) uses $\mathbb{E}_{q(x_i)}[\phi_k]$:

$$(h_{1,r})_k \;=\; \frac{D_{\theta,k}}{\sigma_r^2}\sum_i d_{i,r}\,\mathbb{E}_{q(x_i)}\big[e^{2\pi i\,x_i^\top\xi_k}\big] \;=\; \frac{D_{\theta,k}}{\sigma_r^2}\,\widehat{g_{h_1}}(\xi_k), \qquad g_{h_1} := \sum_i d_{i,r}\,\mathcal{N}(\cdot;m_i,S_i). \tag{13}$$

The Stein correction enters through the adjoint Jacobian. By the derivative rule (10), ∂_{x_j} acts as the multiplier $2\pi i\,\xi_{k,j}$ on the spectral grid (conjugating to form J_ϕ^* flips the sign), so $h_{2,r}$ is the *same* FFT output weighted by C and post-multiplied by $-2\pi i\,\xi_j$:

$$(h_{2,r})_k \;=\; \frac{D_{\theta,k}}{\sigma_r^2}\sum_{j=1}^{D_x}(-2\pi i\,\xi_{k,j})\,\widehat{g_{h_2,j}}(\xi_k), \qquad g_{h_2,j} := \sum_i [C_i]_{j,r}\,\mathcal{N}(\cdot;m_i,S_i). \tag{14}$$

Both are the §2 primitive on the spectral grid — the same object as the generator (12), just at $\{\xi_k\}$ instead of $\{\Delta\xi\}$, with weights $d_{i,r}$ (resp. $[C_i]_{j,r}$) in place of Δ_i/σ_r^2. Each is one deposited density through one FFT; the Jacobian costs only the extra $2\pi i\,\xi_j$ multiplier, not a new spread. Setting $S_i \to 0$ collapses each $\mathcal{N}$ to a δ and recovers the MC build exactly.

5 How much of this is Fourier-specific?

Before the algorithm, it is worth separating what the construction owes to the *features* being Fourier from what it owes to nothing in particular. The split is clean and pinpoints exactly what the basis buys.

The smearing is basis-free. The reframing of §2 — that each expected sufficient statistic is a functional of the *expected* density $g = \sum_i w_i \mathcal{N}(\cdot; m_i, S_i)$ — is pure linearity of expectation and uses nothing about Fourier:

$$\mathbb{E}_q\Big[\sum_i w_i\,\phi_k(x_i)\Big] \;=\; \sum_i w_i \int \phi_k(x)\,\mathcal{N}(x; m_i, S_i)\,dx \;=\; \langle \phi_k,\, g\rangle, \tag{15}$$

for *any* features ϕ_k. "Smear each uncertain point by its posterior covariance, then evaluate features against the smeared cloud" would read identically for random, kernel, or neural features. Even the per-source closed form is not special to Fourier: RBF, arccos/ReLU, and polynomial features all integrate against a Gaussian in closed form — the Gaussian characteristic function is merely the instance for $\phi_k = e^{2\pi i\, x^\top \xi_k}$.

But the smearing is not the sufficient statistics. The smear produces g in *input* space. The expected sufficient statistics A_r, h_r live in *feature* space, and in the Fourier basis they *are* the Fourier coefficients of g: $h_r = \widehat{g_{h_1}}(\xi_k)$ on the spectral grid (13), and A_r's generator $= \widehat{g_T}(\xi_k - \xi_\ell)$ on the difference grid (12). So forming the statistics is *not* the smear; it is the FFT that carries g from input space to feature space. The smear is the FFT's *input*; the statistics are its *output* — different objects related by the transform. (One could instead form them by direct summation of (8), $O(MT)$; the FFT is the fast evaluator of "Fourier coefficients of gridded data," which is what the statistics are.)

Three roles, one basis-free. The pipeline has three steps, and only the first survives a change of features:

Step	Produces	Fourier-specific?
Smear	pseudo-obs density g (input space)	**no** (linearity)
FFT of g	suff. stats A_r-generator, h_r (feature space)	**yes**
Solve $A_r\mu_r = h_r$	the $q(f)$ posterior	**yes** (BTTB)

The FFT step and the BTTB structure of A_r both rest on the *character* property $\phi_k\,\overline{\phi_\ell} = \phi_{k-\ell}$: products of Fourier features are again Fourier features, indexed by the *difference*. This is why $\mathbb{E}_{\phi_k\overline{\phi_\ell}} = \widehat{g}(\xi_k - \xi_\ell)$ depends on (k,ℓ) only through the difference (§3), and it is strictly stronger than "a fast transform exists": even translated-kernel features on a grid ($\phi_k(x) = \kappa(x - z_k)$, which *do* admit an FFT) fail it, because $\mathbb{E}_{\kappa(\cdot - z_k)\,\kappa(\cdot - z_\ell)}$ depends on both indices once q is localised.

Given the pseudo-observations, the Toeplitz structure is the headline. The Toeplitz property does double duty, and the two uses are inseparable. It is what lets A_r be *represented* by an $O(M)$ generator (rather than a dense $O(M^2)$ Gram one would have to materialise) — and that generator *is* the sufficient statistic $\widehat{g_T}$, filled in by one FFT — *and* it is what lets each CG matvec $A_r v$ be an $O(M \log M)$ FFT. "Form the statistic" and "solve fast" are the same Fourier coin.

Cost-wise the solve dominates: forming the statistics is a fixed handful of FFTs per inner iteration ($1 + D_{\text{out}}(1 + D_{\text{lat}})$, §8), while the solve is k_{cg} of them. So, given the smeared pseudo-observations, what the Fourier basis buys is the $O(M)$-storage, $O(M \log M)$-matvec precision.

The clean control: same pseudo-obs, no FFT. SparseGP-SING (`sing/sing.py`) runs the *same* E-step on the *same* smeared pseudo-observations, but in an inducing-point basis: its sufficient statistics are dense Gram blocks formed by direct kernel evaluation (no FFT) and solved by dense/low-rank algebra (no Toeplitz). Identical (15); entirely different cost. That neither the FFT-formation nor the Toeplitz-solve comes from the smearing — both are purchased by choosing Fourier features — is exactly what the EFGP-vs-SparseGP split demonstrates.

6 Algorithm: per-source Gaussian-mixture spreader

The objects we need (§§3–4) are all the §2 primitive (6),

$$F(\xi) \;=\; \widehat{g}(\xi) \;=\; \int g(x)\, e^{2\pi i\, \xi^\top x}\, dx, \qquad g(x) \;:=\; \sum_i w_i\, \mathcal{N}(x;\, m_i, S_i), \tag{16}$$

the continuous Fourier transform of a Gaussian *mixture* density g in physical space (equivalently the per-source CF sum $\sum_i w_i\, e^{2\pi i\, m_i^\top \xi} e^{-2\pi^2 \xi^\top S_i \xi}$, which we never evaluate). This section is how the "$\approx$ FFT" of (6) is actually carried out: build g on a grid, FFT it.

6.1 Spread the densities, FFT, done

The algorithm is an equispaced quadrature of (16) followed by an FFT. On a regular grid $\{x_a\}$ with spacing h_{grid} wide enough to enclose g, the Riemann sum

$$F(\xi) \;\approx\; h_{\text{grid}}^D \sum_a g(x_a)\, e^{2\pi i\, \xi^\top x_a} \tag{17}$$

is, for output frequencies on the dual spectral grid (spacing $h_{\text{spec}} = 1/(N\, h_{\text{grid}})$), exactly a D-dimensional FFT of $g_a := g(x_a)$ up to fixed phase shifts. So the only work is building g_a.

Building g_a is cheap. $\mathcal{N}(x_a; m_i, S_i)$ is negligible past $\sim n_\sigma \sigma_i$ from m_i, so each source contributes only to a stencil of radius $r \approx n_\sigma \sigma_{\max}/h_{\text{grid}}$:

$$g_a \;=\; \sum_{i\,:\, a \in \text{stencil}(i)} w_i\, \mathcal{N}(x_a; m_i, S_i), \tag{18}$$

costing $\sim T(2r+1)^D$ Gaussian evaluations rather than TN^D. So g_a is a sparse stencil accumulator; one FFT then returns $F(\xi_k)$. The spectral grid ($M_{\text{per dim}}^D$) and the BTTB difference grid ($(2M_{\text{per dim}} - 1)^D$) are both centred crops of the same FFT output, so one spread+FFT serves both.

It is a NUFFT with the deconvolution removed. A standard Type-1 NUFFT spreads each point mass with a *fixed artificial* kernel κ (width set by target precision ϵ), FFTs, then *deconvolves* by $\hat{\kappa}$ to undo the spreading. Here the integrand is already a sum of smooth Gaussians, so we spread the *actual* per-source PDFs — one line per stencil cell, $g[a]$ += $w_i\, \mathcal{N}(x_a; m_i, S_i)$ — and *skip*

the deconvolution: the resulting envelope $\hat{\kappa}_i(\xi) = e^{-2\pi^2 \xi^\top S_i \xi}$ is exactly the per-source damping we wanted (the Gaussian CF of §2), not an artefact to remove. Outputs are read directly off the FFT grid, since EFGP's frequencies are themselves gridded. The price for using the true S_i: the stencil radius tracks the data's $\sigma_{\max}$ instead of a fixed ϵ, so heterogeneous $\{S_i\}$ cost more cells (§7).

Two errors. Quadrature (Riemann vs. integral; controlled by $h_{\text{grid}} \lesssim \sigma_{\min}$ and a grid wide enough to hold g) and stencil truncation (dropping cells past r; error $\sim e^{-n_\sigma^2/2}$, quantified next).

6.2 Stencil truncation error

Outside the stencil the Gaussian decays as $\exp(-r^2 h_{\text{grid}}^2/(2\sigma_{\max}^2))$. Truncating at $r = 4\sigma_{\max}/h_{\text{grid}}$ gives relative error $\sim e^{-8} \approx 3 \times 10^{-4}$; matching standard NUFFT precision 10^{-7} requires $r \approx 5.7\sigma_{\max}/h_{\text{grid}}$. In the implementation, r is set conservatively from the largest $\lambda_{\max}(S_i)$ across i.

7 Cost analysis

Per spread call, the work decomposes as

$$\boxed{\text{spread cost} \;=\; T \cdot (2r+1)^D \;+\; N^D \log N.} \tag{19}$$

The FFT term is $O(N^D \log N)$ as in any NUFFT. The spread term is where the cost picture differs from a standard NUFFT.

7.1 The fundamental cost driver

The fine-grid spacing h_{grid} is constrained by two requirements:

1. **Resolution** of the sharpest source: $h_{\text{grid}} \lesssim \sigma_{\min}$ (otherwise the spread under-samples the sharpest Gaussians, which then alias as constants in the FFT).
2. **Spatial coverage**: $N\, h_{\text{grid}} \geq$ source extent $+ \sim 4\sigma_{\max}$ (otherwise wide-tail Gaussians wrap around the periodic FFT boundary).

Picking $h_{\text{grid}} \sim \sigma_{\min}$ to satisfy (1), the stencil radius becomes

$$r \;\sim\; \frac{4\sigma_{\max}}{h_{\text{grid}}} \;\sim\; 4 \cdot \frac{\sigma_{\max}}{\sigma_{\min}}. \tag{20}$$

Per-source cell count $(2r+1)^D \sim (8\sigma_{\max}/\sigma_{\min})^D$.

Comparison to standard NUFFT. A standard NUFFT chooses its kernel width w purely from the desired precision ϵ, independent of data: $(2w+1)^D$ cells per source with $w \sim 7$. Our spreader instead inherits the per-source width from the integrand:

$$\frac{\text{GMIX cost}}{\text{NUFFT cost}} \;\sim\; \left(\frac{\sigma_{\max}/\sigma_{\min}}{w/4}\right)^D. \tag{21}$$

For $D = 2$ and $w = 7$: GMIX matches NUFFT cost when $\sigma_{\max}/\sigma_{\min} \approx 1.75$. When the variance ratio across sources is larger, GMIX pays a $(\sigma_{\max}/\sigma_{\min})^2$ factor over a standard NUFFT.

Implication. GMIX is exact (variant A's only approximation is stencil truncation), but its cost depends on the heterogeneity of $\{S_i\}$ across i. In single-scale regimes (S_i approximately equal), it matches a standard NUFFT. In multi-scale regimes (sharp posteriors at some t, broad at others), it pays a constant-factor cost over MC.

7.2 Optimal stencil radius: $r \approx 8$, fixed

Wall time is *non-monotone* in r: $r=4$ runs the EM in 50 s, $r=8$ in 36 s, $r=16$ in 53 s. Smaller r coarsens the BTTB T_r, inflating the CG iteration count; the optimum balances stencil cost $(2r+1)^D$ against CG work, landing at $r=8$ ($n_\sigma \approx 1.5$) for $D=2$. Adapting r as $\sigma_{\max}$ shrinks during EM does not pay — r is a static JIT argument, so each change forces a ∼10–15 s recompile, more than the ∼5 s it saves. **Default:** pick r once from V_0 at $n_\sigma = 1.5$, floor at 8, freeze; `gmix_adapt_stencil=True` opts into the adaptive variant.

8 Implementation

8.1 New module: `sing/efgp_gmix_spreader.py`

Pure-JAX primitives:

- `_spread_2d(m, S, w, xcen, h_grid, N, r)` — vmap'd per-source stencil evaluation + scatter-add into an $N \times N$ complex grid. Anisotropic Gaussian (uses the full $D \times D$ S_i, including off-diagonals).
- `gmix_nufft_2d(...)` — spread + `ifftshift / ifft2 / fftshift` + Riemann-sum normalisation $(N h_{\text{grid}})^D = (1/h_{\text{spec}})^D$ + centering phase $e^{2\pi i \xi^\top xcen}$.
- `stencil_radius_for(S, h_grid, n_sigma=4.0)` — picks r from $\max_i \lambda_{\max}(S_i)$.
- `pick_grid_size(h_spec, m_extent, sigma_max)` — picks N as the next power of 2 satisfying both resolution and coverage constraints (§7).

8.2 New q(f) update: `compute_mu_r_gmix_jax`

In `sing/efgp_jax_drift.py`, the deterministic (spread+FFT) analogue of the MC `compute_mu_r_jax`:

- Builds the BTTB generator from (12) via one spread+FFT, cropped to the difference grid.
- Builds $h_{1,r}$ and $h_{2,r}$ per output dim r via additional spread+FFTs cropped to the spectral grid.
- Conjugates to align with the existing MC code's iflag=−1 NUFFT convention (so the BTTB matrix and CG solver are unchanged).
- Falls through to the existing `cg_solve` and the rest of the pipeline.

Total spread+FFT calls per inner iter: $1 + D_{\text{out}}(1 + D_{\text{lat}})$ (= 7 for $D_{\text{lat}} = D_{\text{out}} = 2$).

8.3 Driver hook: `estep_method`

`fit_efgp_sing_jax` now takes

- `estep_method` $\in$ {`'mc'`, `'gmix'`}, default `'gmix'`;
- `gmix_fine_N`, `gmix_stencil_r` (auto-picked from V_0 and h_{spec} if omitted, then adapted once after kernel warmup).

All three internal call sites (jit'd inner E-step scan, frozen-qf path, M-step μ-refresh) dispatch on this flag.

8.4 Tests

- `tests/test_gmix_spreader.py` — spreader vs dense closed-form reference on constant-S, varying-S, resolution-convergence, and high-S-variation problems. Errors 10^{-7} to 10^{-4} depending on stencil truncation.
- `tests/test_efgp_gmix_vs_mc.py` — the deterministic `compute_mu_r_gmix_jax` agrees with high-S-marginal MC reference (8 seeds $\times$ $S = 32$) to $\sim$1 %, while single-seed $S = 2$ MC sits at $\sim$12 %. GMIX is $\sim$ 12$\times$ closer to the MC reference than single-seed MC at production settings, confirming it is the unbiased target the MC is estimating.
- `tests/test_gmix_nufft.py` — earlier bucketed-NUFFT correctness/complexity tests (variant B) retained for reference.

9 End-to-end benchmark

9.1 Setup

`demos/bench_gpdrift_mc_vs_gmix.py`: GP-drift recovery in 2D, $T \in \{500, 2000, 20000\}$, $\ell_{\text{true}} = 0.8$, $\sigma^2_{\text{true}} = 1$, $\sigma_{\text{diff}} = 0.3$. Drift sampled fresh from an SE-GP with truth hypers; restoring term $-\alpha x$ added for SDE stability. Gaussian observations through a fixed $C \in \mathbb{R}^{8\times 2}$, $R = 0.05I$; emissions held fixed (`learn_emissions=False`) to isolate the q(f) regression block. Both methods learn kernel hypers from $\ell_{\text{init}} = 0.3$ (0.4$\times$ truth), $\sigma^2_{\text{init}} = 1$. $n_{\text{em}} = 20{-}25$ outer iters, $n_{\text{estep}} = 10$ inner iters. Default GMIX configuration: $n_\sigma = 1.5$ (stencil-truncation eps $\sim e^{-1.1} \approx 0.33$, $\sim$2$\times$ tighter than MC's $S{=}2$ shot noise); no mid-run stencil adaptation; r floored at 8.

9.2 Results — small/medium T

At $T \in \{500, 2000\}$ the methods are close in both quality and wall time; differences are small but consistent in GMIX's favour.

T	method	wall (s)	ℓ/ℓ_{true}	$\sigma^2/\sigma^2_{\text{true}}$	drift_traj	drift_grid	zero-baseline
500	MC	30.9	1.06	0.76	0.148	0.353	0.676
500	GMIX	29.9	1.04	0.71	0.149	0.346	
2000	MC	32.9	1.20	1.06	0.116	0.374	0.716
2000	GMIX	39.9	1.14	0.97	0.111	0.350	

9.3 Results — $T = 20{,}000$

At $T = 20$k the picture splits across seeds. We ran two seeds (controlling the GP-drift draw, the SDE simulation, and the observation noise) plus an MC sample-count sweep. At one seed, MC catastrophically diverges; at another, MC works fine. GMIX is robust on both.

seed	method	wall (s)	ℓ/ℓ_{true}	$\sigma^2/\sigma^2_{\text{true}}$	drift_traj	drift_grid
1	MC, $S=2$	109	1.12	37.81	**5.81**	**6.01**
1	MC, $S=8$	100	0.99	31.03	4.67	4.69
1	MC, $S=16$	118	1.02	28.35	4.43	4.58
1	GMIX	124	1.06	0.71	**0.044**	**0.226**
2	MC, $S=2$	100	1.19	0.87	0.033	0.346
2	MC, $S=8$	109	1.30	1.17	0.034	0.395
2	MC, $S=16$	105	1.29	1.15	0.033	0.390
2	GMIX	114	1.32	1.05	0.031	0.391

Two findings.

1. **MC blow-up at $T = 20$k is not deterministic but is real.** On seed 1 MC's noise compounds through the natural-gradient updates, σ^2 inflates 30–40×, the kernel becomes too aggressive, and the resulting drift posterior is meaningless (drift_traj ~ 5, vs zero-baseline 0.77). On seed 2 the same setup produces a clean fit. We saw the failure on roughly half of attempts at this problem size; smaller T values do not exhibit it.

2. **More MC samples do not rescue the failure.** Going from $S=2$ to $S=8$ to $S=16$ (8× more compute on the sample dimension) on seed 1 nudges drift_traj from 5.81 to 4.43 — still two orders of magnitude worse than GMIX's 0.044. σ^2 remains 28–38× truth. The shot noise compounds through the EM dynamics in a way that is not statistical (more samples per call) but structural (each EM iter accumulates bias from the previous's noisy update).

GMIX has no analogous failure mode: its per-call output is deterministic, so EM iter $k+1$ sees a clean response to iter k's update. Both seeds converge GMIX to within 7% of σ^2_{true} and produce $1-\text{drift\_traj/zero-baseline} \geq 95\%$.

Wall time at $T = 20$k. Within ∼15% of MC across seeds (114–124 s for GMIX vs 100–118 s for MC). The slowdown narrows from ∼1.5× at $T=2$k to ∼1.1× at $T=20$k — the per-iter overhead amortises better at larger T.

9.4 Hyperparameter learning paths and the log-marginal landscape

The hyperparameter paths plotted over the collapsed log-marginal-likelihood landscape (computed under GMIX's converged $q(x)$) appear in `demos/_bench_mc_vs_gmix_out/landscape_paths_T2000_ls0.8.png`. Both paths walk down the same valley to the same neighbourhood; the GMIX trajectory is smoother (no MC noise) and lands slightly closer to the true optimum. At $T=20$k seed 1, MC's path drifts upward toward $\sigma^2 \to \infty$ — visible directly in the path plot.

10 Where this sits among the alternatives

For completeness, a comparison of the closed-form approximations considered earlier:

Variant	Per-spread cost	Error
(A) Custom spreader (this note)	$O(T(\sigma_{\max}/\sigma_{\min})^D + N^D \log N)$	exact (FFT + stencil t
(B) Bucketed NUFFT	$O(BT/B + BM^D \log M) = O(T + BM^D \log M)$	within-bucket bias from
(C) Gauss–Hermite (K^D nodes)	$O(TK^D + M^D \log M)$	deterministic, $\sim (\xi^\top S\xi$
MC (S samples)	$O(TS + M^D \log M)$	stochastic, $\sigma \sim \sqrt{1/S}$

Each row trades a different cost driver for a different error mechanism. Variant A removes both stochasticity and bucket bias at the price of $(\sigma_{\max}/\sigma_{\min})^D$ in spread cost; MC keeps the spread cheap but has shot noise that does not vanish at fixed S.

11 Recommendation

- **Default to GMIX**. It is exact (up to controllable FFT truncation), gives slightly better drift error and hyperparameter recovery, and has no MC noise to interact with the M-step.
- **Use MC if cost matters more than precision.** At $T \gg 1000$ in regimes where $\sigma_{\max}/\sigma_{\min}$ is large, MC remains $\sim 1.5\times$ faster. The MC noise at $S = 2$ does land within the same log-marginal valley; the differences are constant-factor.
- **Tune** `kernel_warmup_iters` so the single GMIX stencil-radius adaptation lands at a sensible $\sigma_{\max}$. Too early and the adaptation triggers before the smoother sharpens; too late and the cold-start runs all use a wide stencil.

12 Math-to-code dictionary

Math	Code
Primitive $F = \widehat{g}$, $\mathbb{E}[\phi]$ (6), (8)	`gmix_nufft_2d`
Generator $T_r = \widehat{g_T}$ (12)	`gmix_nufft_2d` cropped to BTTB diff grid
$h_{1,r} = \widehat{g_{h_1}}$, $h_{2,r}$ (13), (14)	`compute_mu_r_gmix_jax`
Per-source spread $\sum_i w_i \mathcal{N}(\cdot; m_i, S_i)$	`_spread_2d`
Stencil radius from $\sigma_{\max}$	`stencil_radius_for`
Fine grid size N from $h_{\text{spec}} + \sigma$	`pick_grid_size`
Driver flag	`fit_efgp_sing_jax(estep_method=...)`
Adaptive-r once after warmup	`_maybe_adapt_stencil` in `efgp_em.py`
Benchmark	`demos/bench_gpdrift_mc_vs_gmix.py`
Landscape plot	`demos/plot_mc_vs_gmix_landscape.py`
Tests	`tests/test_gmix_spreader.py`, `tests/test_efgp_gmix_vs_mc.py`

None of these touch `drift_moments_jax`, `cg_solve`, `make_A_apply`, or `m_step_kernel_jax`. The custom-VJP shims `ef_with_jac_grad` and `eff_with_grads` are unchanged.